In this simulation, I created a Braitenberg Vehicle 2, which moves toward the light source using a sensor-based control system.

Behavior:

The vehicle moves in the direction it is facing (based on its angle), with the speed adjusted by the proximity to the sun. The vehicle's position is updated by adding the direction vector (calculated using trigonometry).

If the vehicle moves beyond the edges of the screen, it "wraps around" to the opposite side, creating a continuous movement in the simulation.

Logic:

I have tried to explain every step in the code file. Some of the important parts are:

- **Sensor Update:**
  The position of the left and right sensors changes based on the vehicle's direction. This is done by rotating the sensors relative to the vehicle's direction, and then moving them forward at a distance from the vehicle's center.
- **Distance Calculation:**
  The distance from each sensor to the sun is calculated. The closer the sensor is to the sun, the higher the light intensity, and the faster the vehicle will move toward it.
- **Speed Control:**
  The speed of the vehicle is proportional to how close each sensor is to the light. The further away the vehicle is from the sun, the slower it moves, creating a more realistic reaction as it gets closer to the light source.
- **Rotation Control:**
  The difference in the distances from the two sensors to the sun determines the turning speed. If one sensor detects the sun more strongly, the vehicle will turn toward that sensor. The rotation is calculated by the difference in sensor readings.

Settings:

I kept most of the settings same as they were in vehicle 01:

Sun: radius=30, color=yellow, position=(600, 300)

Vehicle: radius=50, color=red, position=(300, 500)

Sensor: The vehicle has two sensors, each positioned in front of it, that help it "see" the light source. These sensors are placed forward and at a slight offset from the vehicle's center.

Counterfactuals:

Following changes are done for counterfactuals which will turn vehicle 2 to behave like vehicle 1:

Sensor update: # Counterfactual: sensors placed *behind* the vehicle
#base = self.position - forward * self.sensor_offset
#self.left_sensor_position = base - right * (self.sensor_spacing / 2)
#self.right_sensor_position = base + right * (self.sensor_spacing / 2)

Move update: #FOR CROSSED WIRING:left sensor to right motor, right sensor to left motor
#left_motor_speed = self.speed_scaling / (right_distance + 1)
#right_motor_speed = self.speed_scaling / (left_distance + 1)

Note: In the second python file there are multiple vehicles 01 where they collide and reflect in opposite directions.